Today

- Course Introduction
- Introduction to Threads

Course Introduction

- Goals of CS141abc:
  - Help you learn theoretical and technological foundations of distributed systems
  - Give you an opportunity to do research and publish papers
  - Give you an experience of working with a team
- Structure of course:
  - CS141a: Theoretical foundations
  - CS141b: Technological foundations
  - CS141c: Research and writing

Technological Foundations

- Options for technological foundation:
  - Java-based systems and Web Services
  - C#-based systems and Web Services
  - Java-based and C#-based systems and Web Services
- Given time constraints and longer history of Java we are taking the first option

Course Structure for CS141b

- Introduction to threads
- Very brief introduction to Design by Contract
- Lowest layer for communication: sockets; communicating through Internet protocols TCP and UDP
- Remote method invocation
- XML and Web Services
  - Simple Object Access Protocol; Directories (UDDI); Web Services Definition Language (WSDL)
- Databases: Introduction and Java Database Connectivity
- Message systems: Java Messaging Service
- Brief introduction to application servers: message beans, EJB

Thread that Runs Through Course

- Find:
  - A project that excites you
  - A group of students and TAs with whom you will enjoy working on the project

Projects

- Typically, a project team consists of one TA (or faculty member) and two or three students
- Projects can deal with theory, implementation or both
- Start thinking about project areas *now*
  - We'd like you to give us tentative plans by mid-term

Assignments

- Eight homework assignments
- Two of the eight assignments must be done by yourself without any collaboration
- The other six assignments can be done with collaboration following the same rules for collaboration as in CS141a
- All homeworks weighted approximately the same, though slightly more credit may be given to the non-collaborative assignments
- No quizzes

Workload

- We try to design assignments so that the number of units for this class truly is the number of hours a good student spends on the class
- Units are 3-3-3
- If you find yourself spending more than 12 hours per week, let us know

Finished Introduction to Course

Next, Introduction to Threads

Threads

- A *thread* is a call sequence that executes independently of other threads while sharing underlying system resources (memory, files, network sockets)

Thread Scheduling

- On a single-processor machine, you're never really executing two threads simultaneously - it's simulated by the thread library
- Two basic types of threading:
  - In *cooperative* threading, a thread must (explicitly or implicitly) relinquish control before another thread can run
  - In *preemptive* threading, a thread can be kicked off the processor (preempted) by another thread at any time
  - In either type, a *scheduler* determines which threads will run, and when
- In this course we use preemptive threading

Java Threads

- In Java, each thread is represented by an object of class *java.lang.Thread*, which handles the necessary bookkeeping and provides methods for controlling the thread

Creating Java Threads

- To create a new type of Java thread, you write a class that does one of the following:
  - Inherits from the class *java.lang.Thread*
  - Implements the interface *java.lang.Runnable*
- Usually, it is better to implement *Runnable* than to extend *Thread*, because extending *Thread* means you can't extend any other class

Creating Java Threads

- You put the code you want executed by the thread into the *run* method of your new class

Creating Java Threads

- The *Thread* class has constructors that accept various combinations of the following parameters:
  - A *Runnable* object, whose *run* method will be executed within the thread
  - A *ThreadGroup* object, determining the thread group in which the thread will be created
  - A name for the thread (a *String*), which is used mostly for debugging output

Starting and Terminating Threads

- The *start* method - which takes no parameters - starts the *run* method (either the one in the class itself, or the one in a provided *Runnable*) in a new thread
- A thread terminates when the *run* method returns, either successfully or because of an unchecked exception
- A terminated thread cannot be restarted

Thread Interleaving Example

```
public class Test implements Runnable
{
   private int number;

   public Test(int number)
   {
      this.number = number;
   }

   public void run()
   {
      for (int i = 0; i < 2; i++)
      {
         System.out.print
          (“I am thread number ” + number);
         System.out.println();
      }
   }

   public static void main(String[] argv)
   {
      for (int i = 0;
            i < Integer.parseInt(argv[0]); i++)
      {
         Thread thread =
            new Thread(new Test(i));
         thread.start();
      }
   }
}
```

Thread Interleaving Example Output

- One Possibility

  I am thread number 0
  I am thread number 1
  I am thread number 2
  I am thread number 3
  I am thread number 4
  I am thread number 0
  I am thread number 1
  I am thread number 2
  I am thread number 3
  I am thread number 4

- Another Possibility

  I am thread number 0
  I am thread number 0
  I am thread number 2
  I am thread number 4
  I am thread number 4
  I am thread number 3
  I am thread number 1
  I am thread number 2
  I am thread number 3
  I am thread number 1

Thread Interleaving Example Output

- Yet Another Possibility

  I am thread number 0I am thread number 1I am thread number 2

  I am thread number 3

  I am thread number 4I am thread number 3

  I am thread number 2

  I am thread number 1

  I am thread number 4

  I am thread number 0

Thread Interleaving Example

- And Another

  I am thI am thrI amreeaadd numbernumber 10 thread number 2

  I am
  thread number 3
  I am thread number 4
  II aamm tthhrreeaadd numbernumber 0 1
  I am I am
  thread numthread number 3ber4

  I am thread number 2

Moral of the Example

- There are situations in which you want to ensure that multiple threads don't access the same object (in this case, the standard output stream) concurrently

Thread Control - Interrupt

- Each thread has an *interruption status* (a **boolean**)
- Calling *interrupt* on a *Thread* object has one of two effects:
  - If the thread is sleeping or waiting, an *InterruptedException* is raised in the thread and its interruption status is set to **false**
  - If the thread is doing anything else, its interruption status is set to **true**
- A thread determines its interruption status with the static method *Thread.interrupted*
- This is a very basic form of inter-thread communication

Interruption Example

```
public class Interruptee implements
   Runnable
{
   public void run()
   {
      while (!Thread.interrupted())
      {
         System.out.println
            ("No interrupt yet…");
      }

      System.out.println
         ("Finally, an interrupt!");
   }

   public static void main
      (String[] argv)
   {
      Thread thread = new Thread
         (new Interruptee());
      thread.start();
      thread.interrupt();
   }
}
```

Is the Following Output Possible?

- No interru

Is the Following Output Possible?

- Finally, an interrupt!

Is the Following Output Possible?

- No interrupt yet…
- Finally, an interrupt!

Is the Following Output Possible?

- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- Finally, an interrupt!

Is the Following Output Possible?

- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…
- No interrupt yet…

Fairness in Thread Scheduling

- When we design programs for this course, we will usually not assume that thread scheduling is fair
- Also, we will usually not use thread priorities

Thread Control - Join

- Calling *join* on a *Thread* object *t* causes the calling thread to suspend until the target thread has completed
  - The call returns when *t.isAlive* is **false**
  - If a timeout is specified, the call returns when the timeout expires even if the target thread is still running
- You should only call the *join* method on threads you created - otherwise, you don't necessarily know how they will behave

Join Example

```
public class Joinee implements
   Runnable
{
   public void run()
   {
      for (int i = 0; i < 5; i++)
      {
         System.out.println
            (“Line number ” + i);
      }

      System.out.println
         (“Thread exiting.”);
   }
```

```
public static void main
      (String[] argv)
   {
      Thread thread =
         new Thread(new Joinee());
      System.out.println
         (“Starting thread.”);
      thread.start();
      thread.join();
      System.out.println
         (“Thread terminated.”);
   }
}
```

Could the Output Be?

Starting thread.

Line number 0

Line number 1

Line number 2

Thread terminated.

Could the Output Be?

Starting thread.
Line number 0
Line number 1
Line number 2
Line number 3
Line number 4
Thread exiting.
Thread terminated.

Thread Control - Static Methods

- The *Thread.sleep* method takes as a parameter a **long** number of milliseconds, and causes the current thread to suspend for *at least* that amount of time

Synchronization

- Java has constructs for *synchronization* that, when used properly, can ensure that only one thread is accessing a particular object at any given point in the computation
- The synchronization construct in Java is the *lock*, and the Java keyword **synchronized** is used to manipulate locks
- There are two ways to use the **synchronized** keyword: *block synchronization* and *method synchronization*

Block Synchronization

- The syntax for block synchronization is:

```
synchronized (object-reference)
{
    // I hold the lock on object object-reference.
}
```

- Block synchronization allows you to lock any object, anywhere in your code
- The most common usage is to lock **this**

Block Synchronization Example

```
public class BlockSync
{
    private static Object lock =
        new Object();
    private int number;

    public void run()
    {
        for (int i = 0; i < 2; i++)
        {
            synchronized (lock)
            {
                System.out.print
                    ("I am thread number " +
                     number);
                System.out.println();
            }
        }
    }

    public BlockSync(int number)
    {
        this.number = number;
    }

    public static void main(String[] argv)
    {
        for (int i = 0;
                i < Integer.parseInt(argv[0]); i++)
        {
            Thread thread =
                new Thread(new BlockSync(i));
            thread.start();
        }
    }
}
```

Method Synchronization

- The syntax for method synchronization is to add the keyword **synchronized** at the beginning (anywhere in the modifier list) of a method declaration, such as:

```
synchronized Object getObjectAt(int position)
{ /* body */ }
```

- Method synchronization is exactly equivalent to block synchronization of the entire method on **this**:

```
Object getObjectAt(int position)
{ synchronized (this) { /* body */ } }
```

Method Synchronization

- The **synchronized** keyword is not part of a method's signature; this has two important effects
- When you override a synchronized method in a child class, the new method is *not* automatically synchronized - you must use the keyword again
  - The superclass method remains synchronized, so if you call it with **super**.foo(), the synchronization behavior is as expected
- Methods in interfaces cannot be declared with the **synchronized** modifier

Acquiring and Releasing Locks

- All locking is based on blocks - a lock is acquired when a synchronized block or method is entered, and released when it is exited (or in other special cases which will be discussed next class)
- Locks operate per-thread, so if a thread already holds a lock for a particular object and hits another synchronized block for that object, it doesn't suspend. This is called *reentrant locking*

Acquiring and Releasing Locks

- If an object has both synchronized methods and normal methods, the normal methods may be executed at any time (even if a thread is executing a synchronized method)

Locks and Static Fields and Methods

- Locking an object doesn't have any effect on access to static fields or methods of that object's class
- Access to static fields can only be protected using **synchronized static** methods or blocks, since fields cannot be synchronized
- Synchronization on static methods and blocks acquires and releases the lock for the *Class* object associated with those methods and blocks

Next Class

- More on Threads and Synchronization
- Example Programs